Invoice

What it is

An invoice lists the services a vendor charged for. The backend reads its line items and compares each one against the vendor's rate card (its committed quotation prices) to work out what the line should have cost, and flags any difference.

This feature is where a reviewer goes through that list: they see the expected price next to the charged price, accept the differences that are fine, and correct the ones the model got wrong.

Routes

Path	Purpose
`/invoice`	The document list: filters, review status, delete
`/invoice/add`	Upload invoice files for a vendor
`/invoice/review`	Review line items (`vendorId` + `documentIds[]`)

`/invoice/review` is a **batch** page — it takes an array of document ids and puts a file switcher in the sidebar. Opening a row from the list always sends a single-element array, so a multi-document review only happens straight after an upload.

Upload

Driven by `use-invoice-submit.ts`. A vendor must be selected — it scopes the rate card the lines will reconcile against.

```
confirm-many → extract-metadata → validate → [invalid-files gate] → [warnings gate]
→ extract-line-items → /invoice/review
```

Step	Endpoint
Link the files to the vendor	`POST /files/confirm-many`
Create documents from files	`POST /files/extract-metadata`
Financial gate + vendor match	`POST /transaction/documents/:id/validate`
Extract + commit line items	`POST /transaction/documents/:id/extract-line-items`

Each step fans out across the batch with `Promise.all`, so the steps are sequential but the documents inside a step are not. As with quotations, `validate` answers 200 and reports rejection in a `status` field rather than a 4xx; rejected files open a dialog and halt the run. Files that validate with warnings let the reviewer continue or discard just the warned documents — unlike quotations, where discard takes the whole batch.

Two differences from the quotation saga are worth understanding:

- `extract-line-items` **commits immediately.** There's no staging step and no `commit-all`.
- **Navigation waits for extraction.** `/invoice/review` needs `documentIds`, and those come from the extract responses — so the page can't be reached until every document in the batch has extracted. Quotations navigate on `vendorId` alone.

Cancel is disabled while the saga runs. The sidebar and browser back remain available as exits.

Matching

Matching is not part of the upload. It fires from an effect when the review page mounts (`use-document-matcher.ts`), for every document in the batch that isn't already matching or matched.

It fires **once per document per session**, guarded by a ref rather than by status. The reason is subtle and documented in the hook: a *synchronous* match returns the final result but never flips the document's status to `matched` — only the queued path's background worker does that. Gating on status would re-fire forever for a synchronously-matched document.

`POST /transaction/documents/:id/match-line-items` answers 201 (synchronous, with counts) or 202 (queued). For the queued case `matchStatusQuery` polls `GET /transaction/documents/:id/match-status` every 2s while the status reads `matching`, and stops the instant it leaves. The review context overlays that live status onto the batch list (fetched once by the loader, never refetched) and invalidates a document's line items on the `matching → matched` edge.

If matching fails, the document stays put and its lines keep reading "Pending".

Data

A line item's match-derived fields — `status`, `reconciliation`, `fromQuotation`, `howDetermined`, `narrative` — are all null until matching runs.

```
type LineItemStatus = "match" | "mismatch" | "needs_review" | "accepted"
```

Labels don't equal wire values: `match` → **Matched**, `mismatch` → **Mismatch**, `needs_review` → **Issue**, `accepted` → **Accepted**. Two client-only display states fill the gap before matching finishes — `processing` while the document is matching, `pending` otherwise — resolved in one place so `isMatching` doesn't have to be threaded to every badge.

`reconciliation` carries `expectedAmount`, `billedAmount`, `difference` and a `direction` (`over` / `under` / `match`). Accepting settles the status but leaves the reconciliation numbers unchanged, so alert tone keys off status, not direction.

Two different statuses, same word

Field	Values	Where
`reviewStatus`	`Pending` / `Completed`	The list's Status **column**
`status` (`DocumentStatus`)	`pending` … `matching` / `matched`	The list's Status **filter**

The column and the filter read **different fields**. The filter groups `DocumentStatus` into two tones (`committed` / `matched` are "success", everything else "pending"); the column shows the server-derived review status. Note also that quotations have a `reviewStatus` too, and it's an unrelated concept with a different type.

Screens

The list

Index, File (name + size), Upload Date, Status (review status), Total Amount, and a row menu whose only item is Delete. Rows open the review page. No sorting.

Vendor and date-range filters are server-side; the date range uses a draft/Apply model rather than committing on each click. **The status filter is client-side over the current page only** — `GET /transaction/documents` takes one exact status, not the tone groups the dropdown offers — so the total and page count keep reflecting the server's vendor+date count, not the status-narrowed one.

Delete is a soft-delete; the API rejects committed documents with a 400.

The review page

A sidebar lists the batch's files with a status dot (red if any line needs attention), a banner counts the lines needing a decision, and tabs filter them: **All**, **Needs Attention** (`mismatch` + `needs_review`), **Matched** (`match` only). Accepted lines are in neither Needs Attention nor Matched — they're reachable only from All.

Search matches the description and invoice ref, nothing else. Sort is default or alphabetical. Both, and the tab, persist when you switch files — which can land you on an apparently empty table.

The detail sheet

Opening a line shows: the Expected · Charged · Difference cells; **From Quotation** (the matched rate-card line — canonical name, tier, attributes, pricing context); **How the result was determined** (the symbols behind the expected amount, then the amounts, with the difference phrased as "Overcharged X" / "Undercharged X" / "Possible shortfall X"); and **Why the AI reached this conclusion?** (the narrative).

Prev/next walks the **filtered and sorted** list in display order — change the tab or the search and the traversal changes under you. If the open line is filtered out, the sheet closes; accepting a line while on the Needs Attention tab does exactly that.

The sheet has two independent actions, each behind its own button and endpoint:

- **Accept** → `POST …/line-items/:id/accept`. Marks the line accepted; it is one-way, with no undo. `acceptedFrom` records the status the line was accepted from.
- **Save guidance** → `POST …/line-items/:id/guidance`. Guidance is a persistent per-line correction re-applied on every rerun.

File index

`features/invoices/`

File	Purpose
`components/invoices-table.tsx`	The list; client-side status filter, delete, row → review
`components/invoices-toolbar.tsx`	Vendor + status selects, date range
`components/date-range-filter.tsx`	Range calendar with draft/Apply
`components/add-invoice-form.tsx`	Upload page: dropzone, vendor select, Extract
`components/delete-invoice-dialog.tsx`	Pure confirm
`components/review/invoice-review.tsx`	Review layout; tab/sort/search state and filtering
`components/review/invoice-review-context.tsx`	The batch, live status overlay, active doc
`components/review/line-item-selection-context.tsx`	Open line, prev/next, accept + guidance
`components/review/review-sidebar.tsx`	File switcher with attention dots
`components/review/review-tabs.tsx`	All / Needs Attention / Matched + counts
`components/review/review-toolbar.tsx`	Sort and search
`components/review/review-table.tsx`	Line-item rows
`components/review/attention-banner.tsx`	Counts mismatches + issues; jumps to the tab
`components/review/line-item-detail-sheet.tsx`	The reconciliation detail surface
`components/review/status-badge.tsx`	Label + tone + icon per display status
`components/review/review-footer.tsx`	Cancel / Complete; owns the mark-reviewed mutation
`components/review/complete-review-dialog.tsx`	Pure confirm listing the files to mark
`hooks/use-invoice-submit.ts`	The upload saga and its gates
`hooks/use-document-matcher.ts`	Fires matching on review-page mount, once per doc
`services/invoices.ts`	Document list, delete, file download (unused)
`services/invoice-upload.ts`	Validate + extract-line-items
`services/invoice-review.ts`	Line items, match, match-status, guidance, accept, mark-reviewed
`lib/invoice-status.ts`	`DocumentStatus` / `ReviewStatus` mirrors + tone

File	Purpose
`lib/line-item-status.ts`	Line statuses, display states, labels, tones, row highlight
`lib/invoice-filters.ts`	The client-side status-tone filter
`lib/invoice-invalid-files-copy.ts`	Copy for the rejected-files dialog
`lib/format-amount.ts`	Currency formatting
`lib/format-document-date.ts`	Date + time split for the list
`store/delete-invoice-dialog.ts`	Delete-confirm open state + target
`types.ts`	Wire types: documents, line items, reconciliation, match results